

Communication & APIs

Why this chapter matters: a system is a set of components that talk to each other — browser to server, service to service, server to client. The **style of communication** you choose (REST, RPC/gRPC, GraphQL, WebSocket, webhooks, queues) decides latency, coupling, scalability and how painful failures are. This chapter builds on Chapter 3: now that you know how bytes travel, we design the *conversation*.

🔗 QUICK RECALL — Chapter map

- 4.1 Communication styles → 4.2 HTTP in depth (methods, status codes, headers, caching) → 4.3 REST → 4.4 RPC & gRPC → 4.5 REST vs RPC
- 4.6 GraphQL → 4.7 Real-time: polling, long polling, SSE, WebSocket, webhooks → 4.8 API design best practices
- 4.9 Making calls reliable (timeouts, retries, idempotency) → 4.10 Choosing a style → 4.11 Practice, Q&A, cheat sheet

4.1 Communication styles

Two big questions decide the shape of any interaction: **Who waits?** and **Who starts the conversation?**

Style	How it works	Coupling	Examples
Synchronous request/response	Caller sends a request and waits for the reply	Tight in time: callee must be up now	REST, gRPC, GraphQL over HTTP
Asynchronous messaging	Caller drops a message on a broker/queue and moves on; consumer processes later	Loose in time; needs a broker	Kafka, RabbitMQ, SQS (Chapter 12)
Server push / streaming	Server sends data to the client without a new request each time	Long-lived connections	WebSocket, SSE, gRPC streaming
Webhook (callback)	Server calls <i>your</i> HTTP endpoint when an event happens	Loose	Stripe/GitHub notifications

Delivery semantics (apply to every network call and message):

Semantics	Meaning	Typical implementation
At-most-once	Sent 0 or 1 times; may be lost, never duplicated	Fire-and-forget, UDP, no retry
At-least-once	Retried until acknowledged; never lost, may be duplicated	Retries, most queues — consumers must be idempotent
Exactly-once	Processed once, no loss, no duplicates	In practice = at-least-once delivery + idempotent processing / deduplication (true end-to-end exactly-once is very hard)

★ MUST REMEMBER

- Over a network you can never be sure whether a timed-out request was executed or not. So: **choose at-least-once and make operations idempotent** — the recurring pattern of this chapter and of distributed systems.

4.2 HTTP in depth

HTTP (Hypertext Transfer Protocol) is a **stateless request/response** application protocol: every request carries everything the server needs, and the server keeps no memory of previous requests (state lives in cookies, tokens, databases). It is self-contained, so requests can flow through proxies, CDNs, load balancers and caches that do load balancing, caching, encryption and compression. It rides on TCP (HTTP/1.1, 2) or QUIC (HTTP/3) — Chapter 3.

Anatomy of a request and a response

```
POST /v1/orders HTTP/1.1          <- request line: method, path, version
Host: api.example.com             <- headers (metadata)
Content-Type: application/json
Authorization: Bearer <token>
Idempotency-Key: 7c9e2a1f-5b7d-4e1c-9a0b-1f2e3d4c5b6a

{"itemId": 456, "qty": 2}          <- body (optional)

HTTP/1.1 201 Created               <- status line: version, code, reason
Location: /v1/orders/9981
Content-Type: application/json

{"id": 9981, "status": "PENDING"}
```

HTTP methods

Method	Meaning	Idempotent*	Safe**	Cacheable
GET	Read a resource	Yes	Yes	Yes
HEAD	Like GET but headers only (check existence/size)	Yes	Yes	Yes
POST	Create a resource or trigger a process that handles data	No	No	Only with explicit freshness info (rare in practice)
PUT	Create or fully replace a resource at a known URI	Yes	No	No
PATCH	Partially update a resource	Not guaranteed	No	Rarely
DELETE	Delete a resource	Yes	No	No
OPTIONS	Ask which methods/headers are allowed (CORS pre-flight)	Yes	Yes	No

* **Idempotent**: calling it many times has the same effect as calling it once (the final state is the same). ** **Safe**: does not change server state (read-only).

◆ SENIOR INSIGHT — Safe vs idempotent — and why it matters

- **DELETE /items/5** twice → the item is gone both times (idempotent, though the 2nd response may be 404). **POST /orders** twice → two orders. Because idempotent methods can be retried automatically by clients, proxies and load balancers, **never hide a state-changing, non-idempotent action behind GET** (crawlers and prefetchers will trigger it).
- **PUT vs PATCH**: PUT sends the whole representation (missing fields are cleared); PATCH sends only changes. A patch like “increment counter” is not idempotent; “set name = X” is.

Status codes

Class	Meaning	Codes to know
1xx	Informational	101 Switching Protocols (WebSocket upgrade)
2xx	Success	200 OK • 201 Created (+ Location header) • 202 Accepted (async, not done yet) • 204 No Content
3xx	Redirect / cache	301 permanent • 302/307 temporary • 308 permanent (keeps method) • 304 Not Modified (cache still valid)
4xx	Client error — fix the request	400 bad request • 401 unauthenticated • 403 forbidden • 404 not found • 405 method not allowed • 409 conflict • 410 gone • 415 unsupported media type • 422 validation failed • 429 too many requests
5xx	Server error — retry may help	500 internal error • 502 bad gateway (invalid reply from upstream) • 503 unavailable/overloaded (+ Retry-After) • 504 gateway timeout (upstream too slow)

★ MUST REMEMBER

- **401** = “I don’t know who you are” (missing/invalid credentials). **403** = “I know who you are, but you’re not allowed.”
- **4xx** = the client should not blindly retry. **5xx** = often safe to retry with backoff (if the request is idempotent). **502/504 usually come from a proxy or load balancer**, not the app itself (Chapter 7).
- Use real status codes — never return 200 OK with {“error”: ...} in the body; it breaks caches, monitoring and client libraries.

Important headers

Header	Purpose
Host	Which site on this IP (virtual hosting)
Content-Type / Accept	Format of the body / formats the client can handle (application/json...)
Content-Length / Transfer-Encoding, Content-Encoding	Body size or chunked streaming; compression (gzip, br)
Authorization	Credentials: Bearer <token>, Basic, API key
Cookie / Set-Cookie	Browser state. Flags: HttpOnly (no JS access), Secure (HTTPS only), SameSite (CSRF defence)
Cache-Control, ETag, If-None-Match, Last-Modified	Caching and conditional requests (below)
Location	Redirect target or URL of newly created resource
Retry-After	How long to wait after 429/503
X-Forwarded-For / Forwarded	Original client IP when behind proxies/load balancers
Idempotency-Key	Client-generated key so the server can de-duplicate retried POSTs (payments APIs)
Access-Control-Allow-*	CORS: which other origins browser JS may call

HTTP caching

Caching is built into HTTP and is the cheapest way to cut latency and server load (CDNs and browsers both obey these headers).

Mechanism	How it works
Cache-Control: max-age=3600	Response is fresh for 1 hour; no request at all during that time

Mechanism	How it works
Cache-Control: public / private	Shared caches (CDN) may store it / only the user's browser may
no-cache vs no-store	<i>no-cache</i> = may store but must revalidate before use. <i>no-store</i> = never store (sensitive data)
ETag + If-None-Match	Server sends a version fingerprint. Client later asks "still this version?"; server answers 304 Not Modified with an empty body → saves bandwidth, not a round trip
stale-while-revalidate	Serve the stale copy instantly while refreshing in the background
Versioned URLs (app.4f9c2.js)	Cache forever (immutable); change the filename to invalidate — solves the "cache invalidation" problem for static assets

◆ SENIOR INSIGHT — CORS in one paragraph

- Browsers block JavaScript on site-a.com from reading responses of api.site-b.com unless the API opts in via Access-Control-Allow-Origin. For non-simple requests (PUT, custom headers, JSON) the browser first sends an **OPTIONS pre-flight**. CORS is a browser rule, not server security: curl and server-to-server calls ignore it. Never use * with credentials.

4.3 REST — Representational State Transfer

REST is an architectural style (Roy Fielding, 2000), not a protocol: the client acts on a set of **resources** managed by the server. The server exposes representations (usually JSON) of those resources, and the client changes them with a small, uniform set of operations (HTTP verbs). All communication is **stateless and cacheable**.

REST constraints

Constraint	Meaning	Benefit
Client-server	UI and data storage are separated	Independent evolution
Stateless	Each request is self-contained; no server-side session memory	Any server can handle any request → easy horizontal scaling
Cacheable	Responses declare whether they can be cached	Lower latency and load
Uniform interface	Resources identified by URIs; manipulated through representations; self-descriptive messages; hypermedia	Simple, decoupled clients
Layered system	Client can't tell if it talks to the origin, a proxy or a CDN	Load balancers, gateways, caches
Code on demand (optional)	Server may send executable code (JavaScript)	—

The four qualities of a RESTful interface (from your notes, refined)

- 1 **Identify resources by URI** — nouns, same URI regardless of operation (/persons/1234).
- 2 **Change them via representations** — verbs, headers and body (GET/POST/PUT/PATCH/DELETE).
- 3 **Self-descriptive messages** — use standard status codes and headers; don't reinvent the wheel.
- 4 **HATEOAS** — *Hypermedia As The Engine Of Application State*: responses include links to what you can do next (e.g., "links": {"cancel": "/orders/9981/cancel"}), so clients follow links instead of hard-coding URLs.

✎ CORRECTION TO YOUR ORIGINAL NOTE — HATEOAS expansion

- Your note says “HTML interface for HTTP”. The acronym actually stands for **Hypermedia As The Engine Of Application State**. It is the highest REST maturity level; most real-world “REST” APIs stop at level 2 (resources + proper verbs + status codes).

Resource design

Good	Avoid	Why
GET /orders/42	GET /getOrder?id=42	Use nouns; the verb is the HTTP method
/users (plural collection)	/user, mixed singular/plural	Consistency
GET /users/7/orders	/users/7/orders/3/items/9/re-funds/1...	Nest at most 1-2 levels; then expose the child at top-level (/items/9)
POST /orders/42/cancel (action sub-resource)	Forcing everything into CRUD	Real business actions that don't map cleanly to CRUD are fine
GET /orders?status=paid&sort=-createdAt	Separate endpoints per filter	Filtering/sorting via query parameters

Sample REST calls

GET	/persons/1234		read one
POST	/persons	{"name": "Asha"}	create -> 201 Created + Location
PUT	/persons/1234	{"name": "Asha K.", ...}	replace the whole resource
PATCH	/persons/1234	{"name": "Asha K."}	partial update
DELETE	/persons/1234		delete -> 204 No Content

◆ SENIOR INSIGHT — Where REST hurts (from your notes, expanded)

- Exposes data, not behaviour:** ‘return all records updated in the last hour matching these events’ is awkward as a path; ends up as query params or a body on POST.
- Few verbs:** ‘move expired documents to the archive folder’ doesn't map cleanly to GET/POST/PUT/DELETE.
- Over-fetching / under-fetching & many round trips:** a blog entry plus its comments plus author may need 3 calls; on slow mobile networks that is painful. Older clients also receive new fields they don't need, bloating payloads.
- Mitigations: ?fields= / ?include=comments, compound endpoints, a **BFF** (Backend-For-Frontend) layer, or GraphQL (4.6).

4.4 RPC & gRPC

In a **Remote Procedure Call**, the client causes a procedure to run in a different address space (usually a remote server), coded **as if it were a local function call**. RPC focuses on exposing **behaviour** (`getUser(id)`, `chargeCard(req)`) rather than resources. Popular frameworks: gRPC/Protobuf, Thrift, Avro (mainly a serialisation format that also defines RPC).

CLIENT SIDE	SERVER SIDE
client program	service procedure (does the real work)
calls func(args)	^ calls with unpacked args
client STUB (marshal: id + args -> bytes)	server STUB (unmarshal, dispatch)
	^
client comm. module (OS sends)	server comm. module (OS receives)

The response travels back through the same steps in reverse order.

- Client stub** marshals (packs) the procedure id and arguments into a message; the **server stub** unmarshals it, calls the matching procedure, and packs the result back.

- The big idea (and danger): it *looks* local but isn't — remote calls are slower, can time out, fail partially, and may execute twice. Treat them as network calls (see 4.9).

gRPC

Feature	Detail
Transport	HTTP/2 (multiplexed, binary, streaming) — long-lived connections reused for many calls
Contract	Protocol Buffers .proto file defines services + messages; code is generated for Go, Java, Python, Node, C++ ...
Streaming	Unary • server-streaming • client-streaming • bidirectional streaming
Built-in	Deadlines/timeouts propagated across services, cancellation, metadata, interceptors, TLS, load-balancing hooks
Strengths	Compact and fast, strongly typed, great for internal microservice calls and polyglot teams
Weaknesses	Not human-readable (harder to curl/debug), limited native browser support (needs gRPC-Web), caching and CDNs less natural, tight coupling to schema

```
syntax = "proto3";
service OrderService { rpc GetOrder (GetOrderRequest) returns (Order); }
message GetOrderRequest { int64 id = 1; }
message Order { int64 id = 1; string status = 2; repeated Item items = 3; }
// Field NUMBERS identify fields on the wire - never reuse or renumber them.
```

Serialisation formats compared

Format	Readable?	Size / speed	Schema	Typical use
JSON	Yes	Larger, slower to parse	Optional (JSON Schema/OpenAPI)	Public REST APIs, web
XML	Yes	Largest, verbose	XSD	Legacy/enterprise, SOAP
Protobuf	No (binary)	Small and fast	Required (.proto)	gRPC, internal services
Avro / Thrift	No (binary)	Small and fast	Required	Kafka/data pipelines (Avro), RPC (Thrift)
MessagePack	No (binary)	Smaller than JSON	None	Drop-in compact JSON

◆ SENIOR INSIGHT — Schema evolution = backward & forward compatibility

- Clients and servers deploy at different times, so changes must not break old peers. Safe: **add optional fields**, ignore unknown fields. Unsafe: remove/rename fields, change types, reuse protobuf field numbers.
- Rule of thumb: **be conservative in what you send, liberal in what you accept**; version the API when a break is unavoidable (4.8).

4.5 REST vs RPC — side by side

Operation	RPC	REST
Signup	POST /signup	POST /persons
Resign	POST /resign { "personid": "1234" }	DELETE /persons/1234
Read a person	GET /readPerson?personid=1234	GET /persons/1234
Read a person's items	GET /readUsersItemsList?personid=1234	GET /persons/1234/items
Add an item to a person's items	POST /addItemToUsersItemsList { "personid": "1234", "itemid": "456" }	POST /persons/1234/items { "itemid": "456" }
Update an item	POST /modifyItem { "itemid": "456", "key": "value" }	PUT /items/456 { "key": "value" }
Delete an item	POST /removeItem { "itemid": "456" }	DELETE /items/456

CORRECTION TO YOUR ORIGINAL NOTE — Typos in the source examples

- The sample payloads in the original reference (and your notes) contain a stray semicolon ("personid": "1234";) — that is **invalid JSON**. Use commas. Also, “modern gRPC” RPC is usually not shaped like GET /readPerson; those are “RPC-style over HTTP”.

	REST	RPC / gRPC
Focus	Resources / data (nouns)	Actions / behaviour (verbs)
Coupling	Loose; generic uniform interface	Tighter; client tied to specific procedures & schema
Performance	Text JSON, good enough; cacheable by CDNs	Binary, compact, fastest (gRPC), streaming
Discoverability / debugging	Easy: curl, browser, standard tooling	Needs tooling (grpcurl), harder to inspect
Caching	Native HTTP caching (GET)	Must be built yourself
Evolution	Add fields/endpoints; version in URL	New RPC per new operation; schema versioning discipline
Best for	Public APIs , browser/mobile clients, CRUD-style domains	Internal service-to-service calls, high throughput, low latency, polyglot, streaming

★ MUST REMEMBER — Rule of thumb

- REST for public / external APIs, gRPC for internal microservice calls** — a heuristic, not a law (Google, Netflix and others expose gRPC and GraphQL publicly too).
- Many systems use **both**: REST/GraphQL at the edge (API gateway) and gRPC between internal services.

4.6 GraphQL

GraphQL is a query language and runtime where the **client asks for exactly the fields it needs** from a single endpoint (usually POST /graphql), against a strongly typed schema.

```
query {  
  order(id: 42) {  
    status  
    customer { name }  
    items { title price }  
  }  
}  
  
response (shape mirrors the query):  
{ "data": { "order": {  
  "status": "PAID",  
  "customer": { "name": "Asha" },  
  "items": [ { "title": "Pen", "price": 10 } ] } } }
```

Strengths	Challenges
No over/under-fetching: one round trip returns exactly the needed shape (great for mobile) Strongly typed schema, introspection, auto-generated docs/clients Evolve by adding fields, deprecating old ones (no /v2) Aggregates many backends behind one API (BFF / federation)	N+1 problem: resolving a list of 100 orders may fire 100 customer queries → fix with batching (DataLoader) HTTP caching is harder (everything is POST to one URL) → persisted queries, client caches (Apollo), CDN tricks Expensive queries: deeply nested queries can DoS you → depth/complexity limits, cost analysis, timeouts, allow-listing Authorisation per field; introspection exposure; more complex server and monitoring (all errors are HTTP 200 + errors array)

◆ SENIOR INSIGHT — Choosing GraphQL

- Great when you have **many different clients** (web, iOS, Android, partners) needing different slices of the same data, or a rapidly changing UI. Overkill for simple CRUD or service-to-service calls.
- Common architecture: clients → **GraphQL gateway/BFF** → internal REST/gRPC microservices.

4.7 Real-time communication: polling, long polling, SSE, WebSocket, webhooks

Plain HTTP is client-initiated. To get updates (chat, notifications, live scores, dashboards, payment results) you need one of these patterns:

Technique	How it works	Pros	Cons	Use for
Short polling	Client asks every N seconds “anything new?”	Trivial; works everywhere	Wasteful, delay up to N s, load grows with clients	Low-frequency status checks (job progress)
Long polling	Server holds the request open until data arrives (or timeout), then client immediately re-asks	Near real-time over plain HTTP; firewall-friendly	One held connection per client; re-connect overhead; ordering care	Fallback for WebSocket, simple notifications
SSE (Server-Sent Events)	One long-lived HTTP response; server streams events one-way (text/event-stream); browser auto-reconnects	Simple, built-in reconnect & event ids, works over HTTP/2	Server → client only; text only; per-browser connection limits on HTTP/1.1	Live feeds, notifications, progress, stock tickers, LLM token streaming
WebSocket	HTTP Upgrade (101) → persistent full-duplex TCP channel; messages both ways with tiny frames	Lowest latency, bidirectional, efficient	Stateful connections are harder to scale & load balance; need heartbeats and reconnect logic; proxies/timeouts	Chat, multiplayer games, collaborative editing, trading UIs
Webhook	Server → server: provider POSTs to your URL when an event occurs	No polling; decoupled	You need a public endpoint; must verify signatures, handle retries/duplicates and ordering	Payment/Git/SaaS events
WebRTC	Peer-to-peer (UDP) audio/video/data using STUN/TURN for NAT traversal	Lowest latency, offloads servers	Complex signalling; TURN relays cost	Video calls, screen share

◆ SENIOR INSIGHT — Scaling WebSockets (a classic design interview topic)

- **Stateful connections:** each client is pinned to one server for hours, so you can't just round-robin per request. Use an L4/L7 load balancer that supports long-lived connections, and keep a **connection registry** (user → server) in Redis.
- **Fan-out across servers:** user A on server 1 sends a message to user B on server 7 → publish to a **pub/sub backbone** (Redis Pub/Sub, Kafka, NATS); every server forwards to its local sockets.
- **Capacity:** a tuned server can hold ~50k-100k+ idle connections (memory per connection and fd limits are the constraint, Chapters 1 & 3). 1 M concurrent users ≈ 10-20+ gateway servers.
- **Reliability:** heartbeats (ping/pong) to detect dead peers; client reconnects with **exponential backoff + jitter** (avoid a thundering herd after a deploy); message ids + ack + resume-from-last-id for no-loss delivery; backpressure for slow consumers.
- **Deploys:** draining 100k connections at once causes a reconnect storm — roll gradually. Messages that must not be lost should go through a durable queue, not only live sockets.

★ MUST REMEMBER — Quick chooser

- **Server → client only** → SSE (or push notifications). **Both directions, low latency** → WebSocket. **Rare checks** → polling. **Server-to-server events** → webhooks or a message queue. **Browser-to-browser media** → WebRTC.

4.8 API design best practices

Pagination — never return unbounded lists

	Offset / limit	Cursor (keyset)
Request	?limit=20&offset=40	?limit=20&after=eyJpZCI6MTIzfQ
SQL	... ORDER BY id LIMIT 20 OFFSET 100000 — the DB reads and throws away 100,000 rows	... WHERE id > :last_id ORDER BY id LIMIT 20 — uses the index, constant cost
Pros	Simple; jump to page N	Fast at any depth; stable when rows are inserted/deleted
Cons	Slow for deep pages; duplicates/skips when data changes between pages	No random page jump; cursor is opaque; needs a unique sortable key
Use	Small datasets, admin UIs	Feeds, infinite scroll, large tables, public APIs

Versioning

Approach	Example	Notes
URI	/v1/orders	Most common, simple, cache-friendly, visible
Header / media type	Accept: application/vnd.myapi.v2+json	Cleaner URLs, harder to test in a browser
Query param	?version=2	Easy but easy to forget

Prefer non-breaking evolution (add optional fields, never remove or change meaning) and version only for real breaks. Publish a **deprecation policy** (Sunset header, notices) and support the old version for a defined period.

Errors, async operations, rate limiting

- **Consistent error format** (RFC 7807 "problem+json"): {"type": "...", "title": "Validation failed", "status": 422, "detail": "qty must be > 0", "traceId": "..."}.
- **Long-running work:** return **202 Accepted** + Location: /jobs/77; the client polls the status resource (or receives a webhook). Never keep an HTTP request open for minutes.

- **Rate limiting:** protect the service and enforce fairness. Respond **429** + Retry-After, expose limit headers. Algorithms: **token bucket** (allows bursts; refill r tokens/s up to capacity b), leaky bucket (smooth output), fixed window, sliding window (more accurate). Usually enforced at the API gateway with Redis counters (Chapter 14).
- **Bulk & partial operations:** batch endpoints reduce round trips; support field selection (`?fields=id,name`) and compression (gzip/br).
- **Security basics:** HTTPS only; authenticate (API keys, OAuth2, JWT — Chapter 13); authorise per resource (prevent IDOR: user A reading `/orders/` of user B); validate input; don't leak internals in errors.
- **Documentation & contract:** OpenAPI/Swagger spec for REST, .proto for gRPC, schema for GraphQL; generate clients and mock servers from it.

4.9 Making calls reliable: timeouts, retries, idempotency

Every remote call can be slow, fail, or succeed without you knowing. Production-grade clients combine these tools:

Tool	What it does	Rules
Timeouts	Bound how long you wait (connect timeout + request timeout)	Always set them; “infinite” defaults cause thread/connection pile-ups. Set from the SLO (e.g., p99 + margin)
Retries	Re-send after transient failure (network error, 502/503/504, 429)	Only for idempotent operations (or with an idempotency key); limit attempts (2–3); never retry 4xx validation errors
Exponential backoff + jitter	Wait longer after each failure, with randomness	$\text{sleep} = \text{random}(0, \min(\text{cap}, \text{base} \times 2^{\text{attempt}}))$. Jitter prevents synchronised retry storms
Idempotency keys	Client sends a unique key per logical operation; server stores the result and replays it on duplicates	Essential for POST /payments, orders, emails
Circuit breaker	After many failures, stop calling the sick dependency for a while (open) and fail fast; probe occasionally (half-open)	Protects both caller and callee; pair with fallbacks
Deadline propagation	Pass the remaining time budget downstream (gRPC deadlines)	Prevents doing work for a caller that already gave up
Bulkheads & load shedding	Isolate resource pools per dependency; reject excess work early (503)	One slow dependency can't exhaust all threads

Idempotent POST with an Idempotency-Key (Node/Express sketch)

```
app.post('/payments', async (req, res) => {
  const key = req.get('Idempotency-Key');
  if (!key) return res.status(400).json({ error: 'Idempotency-Key header required' });

  const done = await redis.get(`idem:${key}`); // already processed?
  if (done) return res.status(200).json(JSON.parse(done)); // replay the SAME result

  const locked = await redis.set(`idem:${key}:lock`, '1', { NX: true, EX: 30 });
  if (!locked) return res.status(409).json({ error: 'request in progress' });

  const result = await chargeCard(req.body); // side effect happens once
  await redis.set(`idem:${key}`, JSON.stringify(result), { EX: 86400 });
  res.status(201).json(result);
});
```

✖ COMMON MISTAKE — Retry storms & amplification

- If each of 3 layers retries 3 times, one user request can become **27** calls hitting an already struggling database. Retry at **one layer** (usually the edge/client), use budgets (e.g., retries $\leq 10\%$ of traffic), backoff with jitter, and circuit breakers.
- A timeout does **not** mean the server did nothing. That is why unsafe retries without idempotency cause double charges and duplicate emails.

4.10 Choosing a communication style

Need	Choose	Why
Public API, many third-party developers, CRUD-ish domain	REST + JSON (OpenAPI)	Familiar, cacheable, easy to debug, huge tooling
Internal microservices, high QPS, low latency, strict contracts	gRPC	Binary, HTTP/2, codegen, streaming, deadlines
Multiple UIs needing different slices of data; mobile with slow networks	GraphQL (often as BFF)	One round trip, exact fields
Server pushes updates to browsers (one-way)	SSE	Simple, auto-reconnect
Chat, games, collaboration (two-way, low latency)	WebSocket	Full-duplex persistent channel
Work that can be done later / must survive failures / spike absorption	Message queue / event stream (Chapter 12)	Decouples in time, buffers load, retries
Notify external systems of events	Webhooks (+ signature, retries)	No polling by the partner
Audio/video between users	WebRTC	Peer-to-peer UDP

4.11 Practice questions (try before looking)

P1. Which of GET, POST, PUT, DELETE are idempotent? Which are safe?

Answer: Idempotent: **GET, PUT, DELETE** (POST is not). Safe (read-only): **GET** only.

P2. A client times out calling POST /payments and retries. How do you avoid charging twice?

Answer: Client sends an **Idempotency-Key**; the server stores the result under that key and returns the stored result for duplicates (with a lock for in-flight requests).

P3. Users jump to page 50,000 of a 10-million-row list using ?offset=. Why is it slow and what do you do?

Answer: The DB scans and discards 1,000,000 rows (offset 50,000 $\times$ 20). Use **cursor/keyset pagination**:
WHERE id > :last ORDER BY id LIMIT 20 on an index.

P4. Chat app with 1 M concurrent users, ~50k connections per gateway server. Servers needed and how do messages reach users on other servers?

Answer: $1\text{ M} \div 50\text{k} = \mathbf{20\ servers}$ (+ headroom). Use pub/sub (Redis/Kafka) so each server forwards to its local sockets; a registry maps user $\rightarrow$ server.

P5. Pick status codes: new order created • user not logged in • logged in but not owner • duplicate email • too many requests • upstream took too long.

Answer: **201 • 401 • 403 • 409 • 429 • 504**.

Interview Q&A (30-second answers)

Q1. REST vs gRPC — how do you choose?

REST/JSON for public, browser and CRUD-style APIs (cacheable, debuggable). gRPC for internal, latency-sensitive, strongly-typed service-to-service calls with streaming needs.

Q2. What makes an API 'RESTful' and what does stateless mean?

Resources identified by URIs, manipulated via standard verbs and representations, cacheable, layered, and each request contains all context — no server-side session — so any instance can serve it.

Q3. PUT vs PATCH vs POST?

POST creates/triggers (not idempotent). PUT replaces a resource at a known URI (idempotent). PATCH partially updates (idempotency depends on the patch semantics).

Q4. 401 vs 403?

401 = not authenticated (who are you?). 403 = authenticated but not authorised.

Q5. How does GraphQL solve over-fetching and what new problems appear?

Client selects fields in one query. New issues: N+1 resolvers (use DataLoader), harder HTTP caching, expensive/malicious queries (limit depth/cost).

Q6. WebSocket vs SSE vs long polling?

WebSocket: bidirectional, lowest latency, stateful. SSE: server→client stream over HTTP with auto-reconnect. Long polling: HTTP-only fallback with held requests.

Q7. How do you make a POST safely retryable?

Idempotency key + stored result (and a lock), or natural idempotency via PUT with client-generated ids; dedupe on the server.

Q8. How would you version an API without breaking clients?

Evolve compatibly (add optional fields), version in the URI for real breaks, run versions in parallel, publish deprecation timelines and monitor old-version usage.

Q9. What are ETags?

Version fingerprints for conditional requests: If-None-Match returns 304 with no body when unchanged — saves bandwidth and server work.

Q10. Why set timeouts and use jittered backoff?

Without timeouts a slow dependency exhausts threads/connections; without jitter clients retry in synchronised waves and prolong the outage.

Chapter 4 Cheat Sheet

◀ QUICK RECALL — Revise this in 2 minutes

- **HTTP:** stateless request/response. Methods: GET (safe, idempotent), POST (neither), PUT (idempotent, replace), PATCH (partial), DELETE (idempotent).
- **Status:** 2xx ok (201 created, 202 accepted, 204 empty) • 3xx redirect (304 cache valid) • 4xx client (400, **401 authN**, **403 authZ**, 404, 409, 422, 429) • 5xx server (500, 502, 503, 504).
- **Caching:** Cache-Control max-age, ETag/If-None-Match → 304, no-store for secrets, versioned URLs for static files.
- **REST:** resources (nouns) + verbs + status codes, stateless, cacheable; HATEOAS = hypermedia links. Pain: round trips, over-fetching.
- **RPC/gRPC:** actions, Protobuf over HTTP/2, codegen, streaming, deadlines; best for internal services. Remote ≠ local: can fail/duplicate.
- **GraphQL:** one endpoint, client picks fields; watch N+1 (DataLoader), caching, query cost.
- **Real-time:** polling → long polling → SSE (one-way) → WebSocket (two-way, stateful; scale with pub/sub) • webhooks for server-to-server events.
- **Pagination:** cursor/keyset beats offset at scale. **Versioning:** additive changes first, /v1 for breaks.
- **Reliability:** timeouts everywhere, retry only idempotent ops, exponential backoff + **jitter**, idempotency keys, circuit breaker, one retry layer.
- **Delivery:** at-most-once / **at-least-once + idempotent consumer** / (effectively) exactly-once.
- **Choosing:** public CRUD → REST • internal fast → gRPC • flexible UI data → GraphQL • live push → SSE/WebSocket • async/durable → queues.